# HW5

Nathan Krieger

2026-03-03

## Problem 1: Best subset selection

The data for this problem comes from a study by Stamey et al. (1989). They examined the relationship between the level of prostate-specific antigen and a number of clinical measures in men who were about to receive a radical prostatectomy. The variables are log cancer volume (lcavol), log prostate weight (lweight), age, log of the amount of benign prostatic hyperplasia (lbph), seminal vesicle invasion (svi), log of capsular penetration (lcp), Gleason score (gleason), and percent of Gleason scores 4 or 5 (ppp45). The last column corresponds to which observations were used in the training set and which were used in the test set (train).

Read in the prostate data set using the following code:

```
prostate = read.table('~/git/DS301/HW5/prostate.data',header=TRUE)
```

Our response of interest here is the log prostate-specific antigen (lpsa). We will use this data set to practice 3 common subset selection approaches.

### (a)

Approach 1: Perform best subset selection on the entire data set with lpsa as the response. For each model size, you will obtain a 'best' model (size here is just the number of predictors in the model): M1 is the best model with 1 predictor (size 1), M2 is the best model with 2 predictors (size 2), and so on.

Create a table of the AIC, BIC, adjusted R2 and Mallow's Cp for each model size. Report the model with the smallest AIC, smallest BIC, largest adjusted R2 and smallest Mallow's Cp. Do they lead to different results? Using your own judgement, choose a final model.

```
set.seed(123)

library(leaps)
regfit <- regsubsets(lpsa ~ ., data = prostate, nbest = 1, nvmax = 8)

n = dim(prostate)[1]

regfit
```

```
## Subset selection object
## Call: regsubsets.formula(lpsa ~ ., data = prostate, nbest = 1, nvmax = 8)
## 9 Variables  (and intercept)
##           Forced in Forced out
## lcavol        FALSE      FALSE
## lweight       FALSE      FALSE
## age           FALSE      FALSE
## lbph          FALSE      FALSE
## svi           FALSE      FALSE
## lcp           FALSE      FALSE
```

```
## gleason       FALSE      FALSE
## pgg45         FALSE      FALSE
## trainTRUE     FALSE      FALSE
## 1 subsets of each size up to 8
## Selection Algorithm: exhaustive
```

```r
regfit_summary <- summary(regfit)

adjr2 <- regfit_summary$adjr2

rss <- regfit_summary$rss

p <- rowSums(regfit_summary$which)

aic <- n*log(rss / n) + 2*p

bic <- n*log(rss / n) + p*log(n)

cp <- regfit_summary$cp


best_adjr2 <- which.max(adjr2)
best_cp <- which.min(cp)
best_aic <- which.min(aic)
best_bic <- which.min(bic)

best_aic
```

```
## 5
## 5
```

```r
best_bic
```

```
## 3
## 3
```

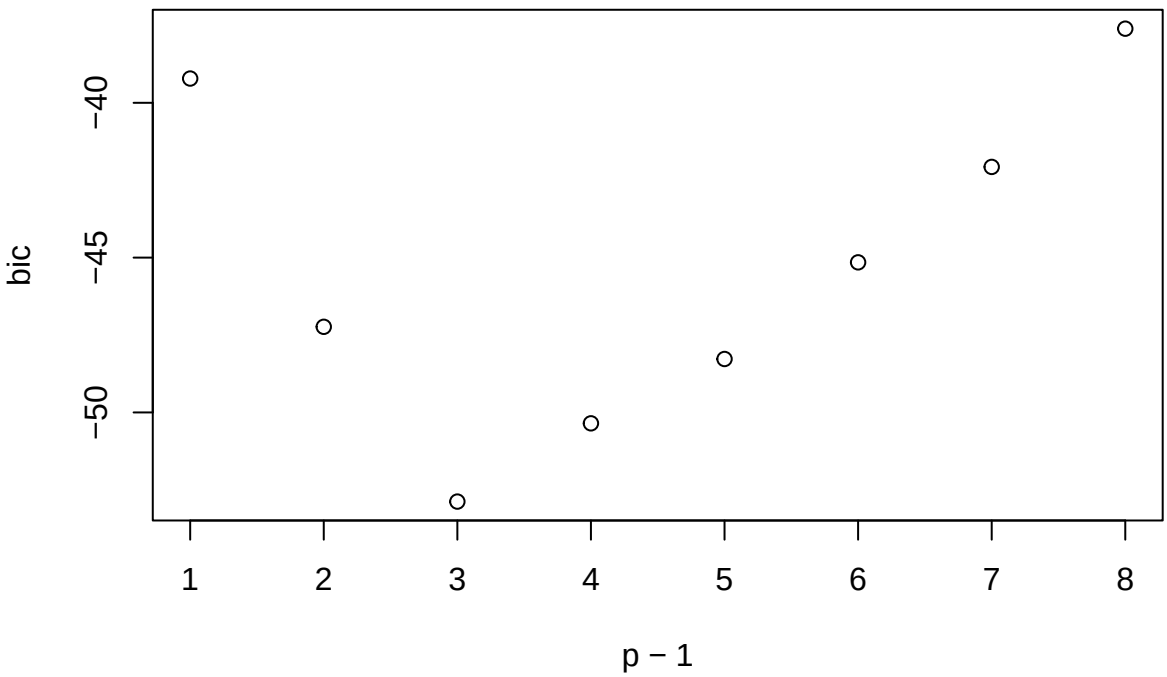
```r
best_adjr2
```

```
## [1] 7
```

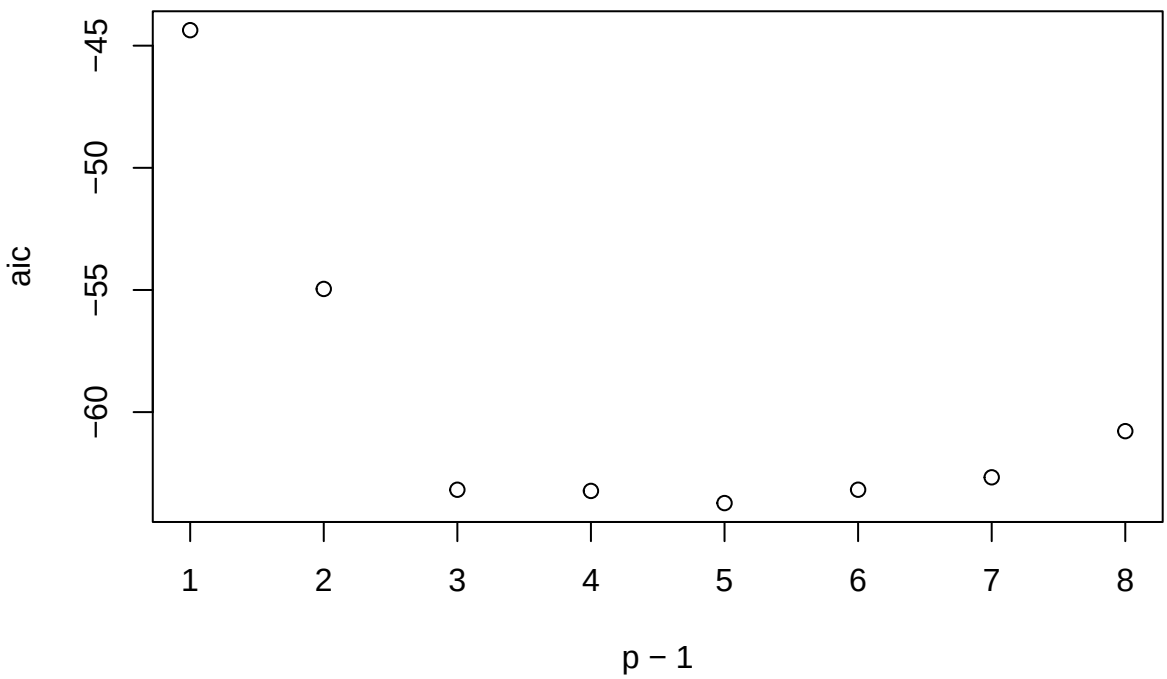
```r
best_cp
```

```
## [1] 5
```

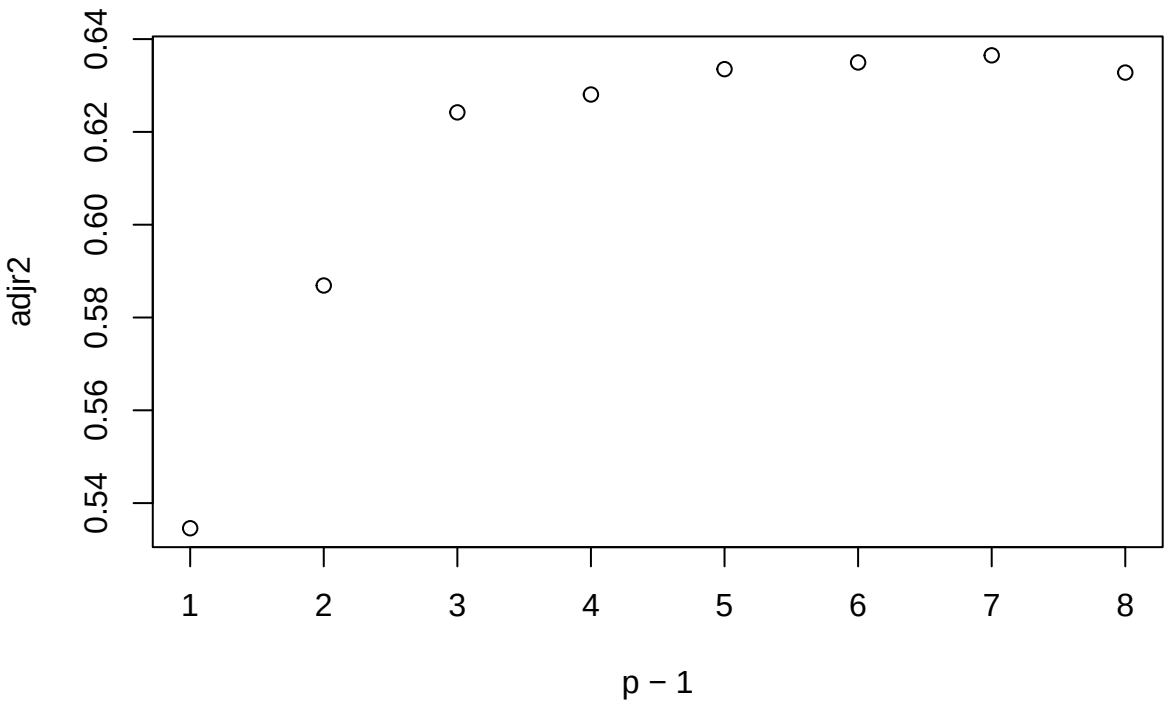
```r
plot(p - 1, bic)
```



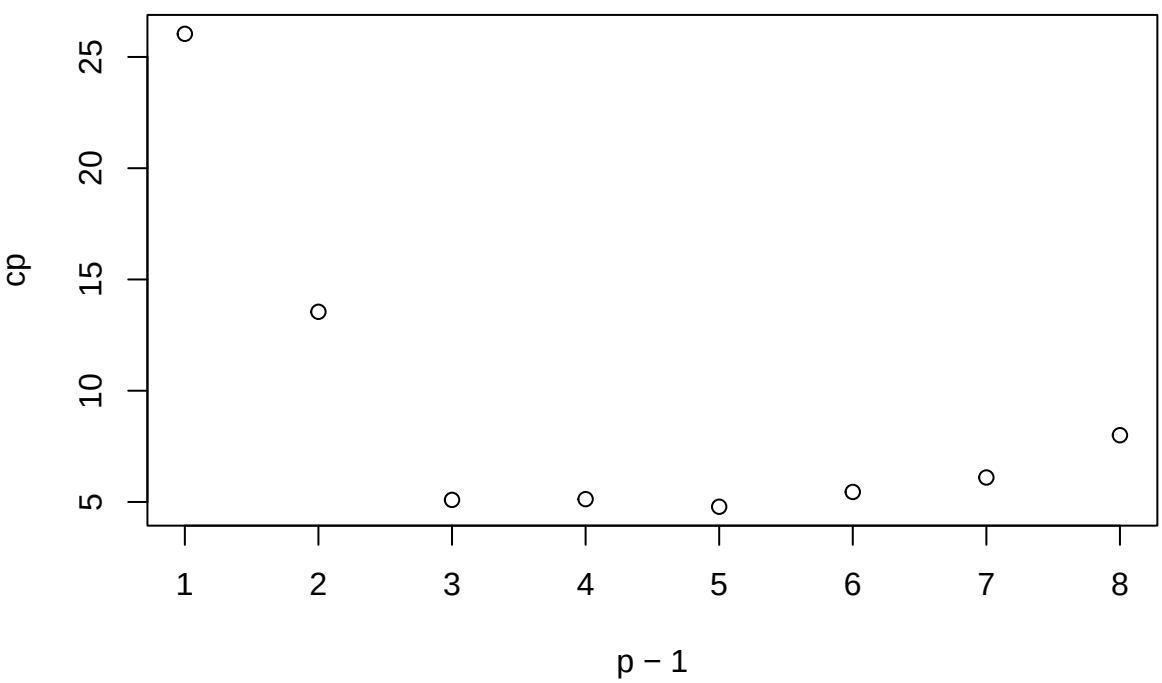

```r
plot(p - 1, aic)
```




```r
plot(p - 1, adjr2)
```




```r
plot(p - 1, cp)
```




```r
cbind(p, rss, adjr2, cp, aic, bic)
```

```
##   p      rss     adjr2        cp       aic       bic
## 1 2 58.91478 0.5345839 26.038827 -44.36603 -39.21661
## 2 3 51.74218 0.5868977 13.546389 -54.95846 -47.23433
## 3 4 46.56844 0.6242063  5.092716 -63.17744 -52.87859
## 4 5 45.59547 0.6280585  5.126817 -63.22555 -50.35199
## 5 6 44.43668 0.6335279  4.785451 -63.72263 -48.27437
```

```
## 6 7 43.77597 0.6349654  5.450474 -63.17571 -45.15273
## 7 8 43.10756 0.6365002  6.099923 -62.66823 -42.07054
## 8 9 43.05842 0.6327886  8.000636 -60.77886 -37.60646
```

```r
cbind(p, rss, adjr2, cp, aic, bic)[5, ]
```

```
##           p         rss       adjr2          cp         aic         bic
##   6.0000000  44.4366818   0.6335279   4.7854515 -63.7226313 -48.2743655
```

Best AIC is 5

Best BIC is 3

Best adr2 is 7

Best CP is 5

Using my best judgement, I believe model size 5 is the best. This is because it has the ideal AIC and CP and the adjr2 and BIC are not far from the best.

## (b)

Approach 2: The dataset has already been split into a training and test set. Construct your training and test set based on this split. You may use the following code for convenience:

```r
train = subset(prostate,train==TRUE)[,1:9]

test = subset(prostate,train==FALSE)[,1:9]
```

For each model size, you will obtain a 'best' model. Fit each of those models on the training set. Then evaluate the model performance on the test set by computing their test MSE.

Choose a final model based on prediction accuracy. Fit that model to the full dataset and report your final model here.

```r
best_train <- regsubsets(lpsa ~ ., data = train, nbest = 1, nvmax = 8)

## the predict() function does not work on regsubsets objects,
# so we need to write our own code to obtain our predicted values:

# Convert the test dataset into a model matrix (needed for matrix multiplication)
# This creates a numeric matrix with an intercept column and all predictor variables

test_mse <- rep(NA, 8)
test_mat <- model.matrix(lpsa ~ ., data = test)
# Loop through all 19 possible model sizes
for (i in 1:8) {

  # Extract the coefficients for the model with 'i' predictors from the best subset selection
  coef_m <- coef(best_train, id = i)
  # Compute the predicted values for the test set using matrix multiplication
  # Multiply the selected predictors' values in 'test.mat' by their corresponding coefficients

  pred <- test_mat[ , names(coef_m)]%*%coef_m

  # Calculate MSE
  test_mse[i] <- mean((pred - test$lpsa)^2)
}
```

```r
test_mse
```

```
## [1] 0.4797387 0.4924823 0.4005308 0.4563321 0.4859242 0.5485933 0.5165135
## [8] 0.5212740
```

```r
# Identify the best model size based on the minimum testing error
optimal_size <- which.min(test_mse)
optimal_size
```

```
## [1] 3
```

```r
# Train the final model using the optimal number of predictors
final_model <- regsubsets(lpsa ~ ., data = prostate, nvmax = optimal_size)

# Get the selected predictors for the final model
final_coeffs <- coef(final_model, id = optimal_size)
knitr::kable(final_coeffs)
```

| | x |
|---|---|
| (Intercept) | -0.7771566 |
| lcavol | 0.5258519 |
| lweight | 0.6617699 |
| svi | 0.6656666 |

Final model has 3 predictors: lcavol, lweight, svi

## (c)

Approach 3 (practice): This approach is used to select the optimal size, not which pre- dictors will end up in our model. Split the dataset into k folds (you decide what k should be). We will perform best subset selection within each of the k training sets. Here are more detailed instructions:

### i. (practice)For each fold k = 1, . . . , K:

1. Perform best subset selection using all the data except for those in fold k (training set). For each model size, you will obtain a 'best' model.
2. For each 'best' model, evaluate the test MSE on the data in fold k (test set).
3. Store the test MSE for each model.

Once you have completed this for all k folds, take the average of your test MSEs for each model size. In other words, for all k models of size 1, you will compute their k- fold cross-validated error. For all the k models of size 2, you will compute their k-fold cross-validated errors, and so on. Report your 8 CV errors here.

```r
library(caret)
```

```
## Loading required package: ggplot2

## Loading required package: lattice
```

```r
set.seed(10)

# Number of folds for cross-validation
k = 10
flds = createFolds(prostate$lpsa, k = k, list = TRUE)  # Create folds
```

```r
# Number of predictors
max_models = 8

# Matrix to store cross-validation errors for each model size across all folds
cv_errors = matrix(NA, nrow = k, ncol = max_models)

# Outer loop: Iterate through each fold
for (i in 1:k) {
  # Get test indices for this fold
  test_index = flds[[i]]
  # Split the dataset into training and validation sets for the current fold:
  test_data = prostate[test_index, ]  # Validation set
  train_data = prostate[-test_index, ]  # Training set

  # Perform Best Subset Selection on the training set
  regfit = regsubsets(lpsa ~ ., data = train_data, nbest = 1, nvmax = max_models)

  # Inner loop: Iterate through each model size (1 to max_models)
  # Calculate cv error (test MSE)
  for (j in 1:max_models) {
    test.mat = model.matrix(lpsa ~ ., data = test_data)
    coef.m = coef(regfit, id = j)
    pred = test.mat[, names(coef.m)] %*% coef.m
    cv_errors[i, j] = mean((test_data$lpsa - pred)^2)
  }

}

# Compute the average cross-validation error for each model size
mean_cv_errors = colMeans(cv_errors)
knitr::kable(mean_cv_errors)
```

| x |
|---|
| 0.6404558 |
| 0.6211504 |
| 0.5265414 |
| 0.5559975 |
| 0.5782019 |
| 0.5748283 |
| 0.5558039 |
| 0.5588542 |

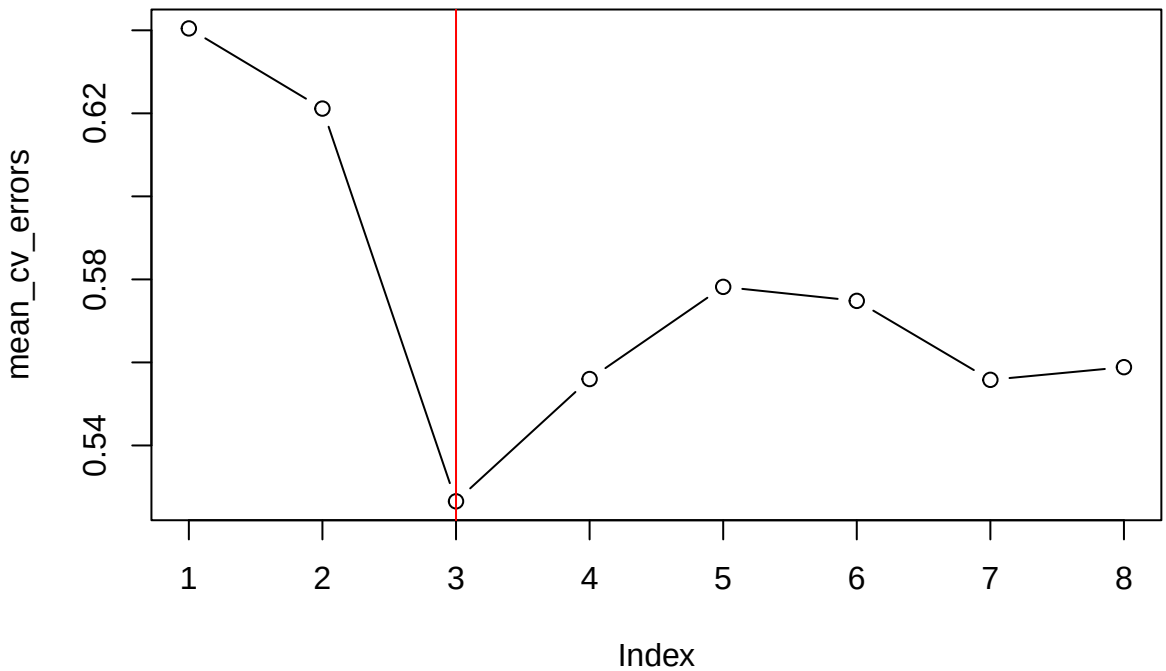
### ii. (practice)

Choose the model size that gives you the smallest CV error. Now perform best subset selection on the full data set again in order to obtain this final model. Report that model here. (For example, suppose cross-validation selected a 5-predictor model. I would perform best subset selection on the full data set again in order to obtain the 5-predictor model.)

```r
# Identify the best model size based on the minimum cross-validation error
optimal_size = which.min(mean_cv_errors)
print(paste("Optimal model size:", optimal_size))
```

```
## [1] "Optimal model size: 3"
```

```r
# Plot cross-validation errors to visualize model performance
plot(mean_cv_errors, type = "b")
abline(v = optimal_size, col = "red")
```




```r
# Train the final model using the optimal number of predictors
final_model <- regsubsets(lpsa ~ ., data = prostate, nvmax = optimal_size)

# Get the selected predictors for the final model
final_coeffs <- coef(final_model, id = optimal_size)
print("Final Model Coefficients:")
```

```
## [1] "Final Model Coefficients:"
```

```r
knitr::kable(final_coeffs)
```

| | x |
|---|---|
| (Intercept) | -0.7771566 |
| lcavol | 0.5258519 |
| lweight | 0.6617699 |
| svi | 0.6656666 |

## Problem 2: Concept Review

### (a)

For the following statement, state whether or not it is True or False. Briefly justify your answer.

Suppose I have 3 models to pick from:

MA :Y ~ X1 + X2 + X3 + X4 + X5

MB :Y ~ X6 + X7 + X8 + X9 + X10

MC :Y ~ X1 + X2 + X7 + X9 + X10

Using AIC, BIC, Mallow's Cp, adjusted R2 could lead us to pick different final models.

False.

Because p and n are completely identical across MA, MB, and MC, the penalty term for each metric becomes a fixed constant for all three models.

Whichever model fits the data the best will simultaneously minimize AIC, minimize BIC, minimize Mallow's Cp, and maximize Adjusted R2. They cannot disagree when comparing models of the exact same size.

### (b)

For the following statement, state whether or not it is True or False. Briefly justify your answer.

Suppose I have two models:

M3 :Y ~ X1 + X2 + X4

M4 :Y ~ X4 + X5 + X6 + X7

It must be that RSS3 >= RSS4.

False. These models are not nested. The only thing they have in common is X4. RSS behavior is not guaranteed since one model is not a subset of the other.

### (c)

For the following statement, state whether or not it is True or False. Briefly justify your answer.

Suppose I have two models:

M2 :Y ~ X4 + X5

M4 :Y ~ X4 + X5 + X6 + X7

It must be that RSS2 >= RSS4.

True. These are nested models. M2 is a subset of M4. When more predictors are added the RSS will either stay the same or decrease.

### (d)

(practice) What advantages are there to using AIC/BIC instead of using the test MSE as

We don't need to split the dataset. It uses the whole thing.

More stable

Computational cheaper than CV

## Problem 3: Diagnostics for MLR (practice)

(a) List the four assumptions we make when fitting a multiple linear regression model.

1. Relationship between Y and X = (X1, X2, . . . , Xp ) is approximately linear.
2. $E(\epsilon) = 0$.
3. $Var(\epsilon) = \sigma^2$.
4. $\epsilon$'s are uncorrelated.

(b) True or False? In order to fit a multiple linear regression model (i.e. obtain least square estimates), we must have distributional assumptions on the random error term epsilon i. Briefly justify your answer.

False. You do not need any distributional assumptions on the random error term. There is no need for distributional assumptions when getting lease squares estimates. Distributional assumptions are only required later if you want to perform statistical inference, such as hypothesis testing or creating confidence intervals.

(c) True or False? The summary(lm()) output automatically gives you p-value for individual hypothesis tests: H0 : beta_j = 0 versus H1 : beta_j = 0, j = 1, . . . , p. These hypothesis tests assume that the random error term are normally distributed. Briefly justify your answer.

True. The summary provides the hypothesis tests in the Pr(t) column. The test statistic assumes that the error term is normally distributed.

(d) Suppose we want to run some visual diagnostics to check whether or not the linearity as- sumption holds. Besides the fact that it is is time consuming, explain why it is not sufficient to check every pairwise scatterplot between Y and Xj , j = 1, . . . , p.

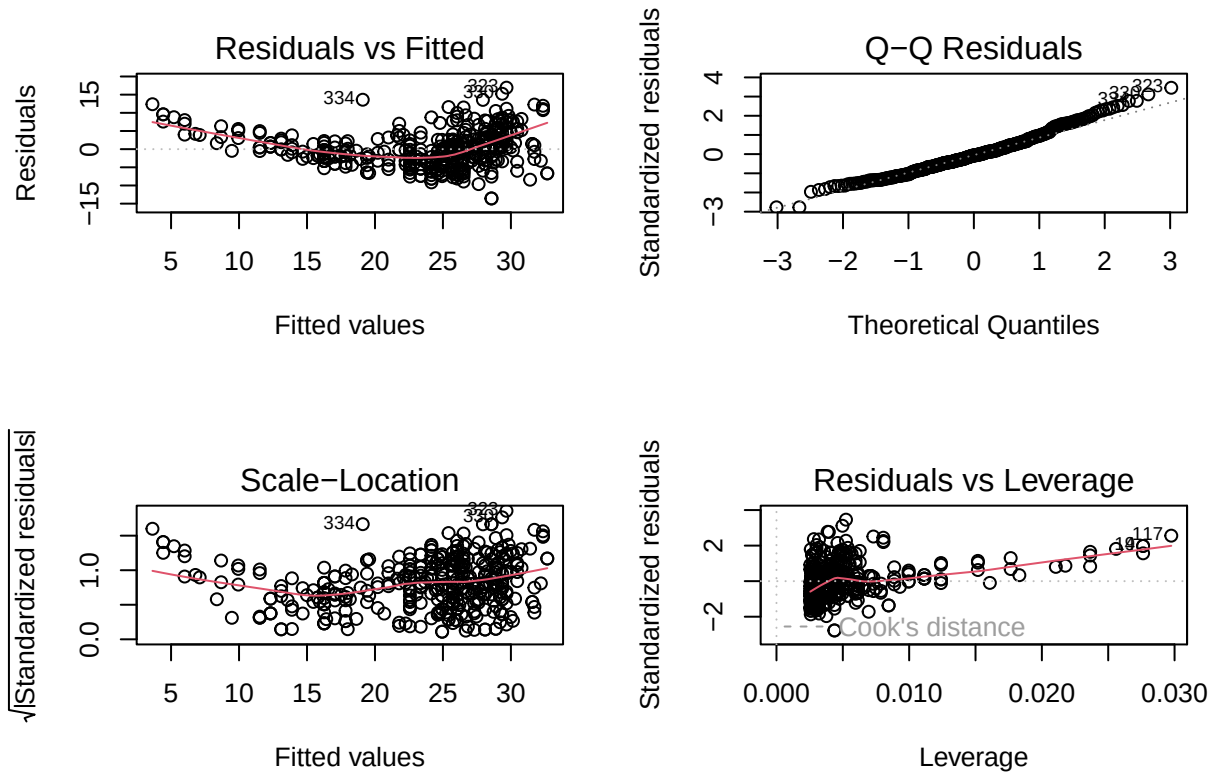
Checking the marginal scatterplot between Y and a single predictor Xj only shows you the marginal (unadjusted) relationship between those two variables, completely ignoring the other predictors in your model.

(e) To check whether or not the linearity assumption holds, we can run some diagnostics. Let's use the Auto dataset as an example from library(ISLR2). Run the following code:

```r
library(ISLR2)
m1 = lm(mpg~horsepower,data=Auto)
summary(m1)
```

```
##
## Call:
## lm(formula = mpg ~ horsepower, data = Auto)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.5710  -3.2592  -0.3435   2.7630  16.9240
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 39.935861   0.717499   55.66   <2e-16 ***
## horsepower  -0.157845   0.006446  -24.49   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.906 on 390 degrees of freedom
## Multiple R-squared:  0.6059, Adjusted R-squared:  0.6049
## F-statistic: 599.7 on 1 and 390 DF,  p-value: < 2.2e-16
```

```r
par(mfrow=c(2,2))
plot(m1)
```




Take a look at the plot in the top left corner (titled Residuals vs Fitted). This is what we call a residual plot because it is literally a plot of residuals versus fitted values (ˆyi). The red line is a smooth fit to the residuals, which is displayed in order to make it easier to identify any trends. You should observe that the plot exhibits a clear U-shape, which provides a strong indication of non-linearity in the data.

Ideally what we want to see is no discernible pattern in the residual plot and a relatively flat red line. A residual plot exhibiting no pattern and a flat red line indicates that the linearity assumption for our model holds. Propose a more flexible model than m1 that might be able to accommodate some non-linearity. Present your proposed model (call it m2) and its corresponding residual plot here. Is it an improvement over m1?

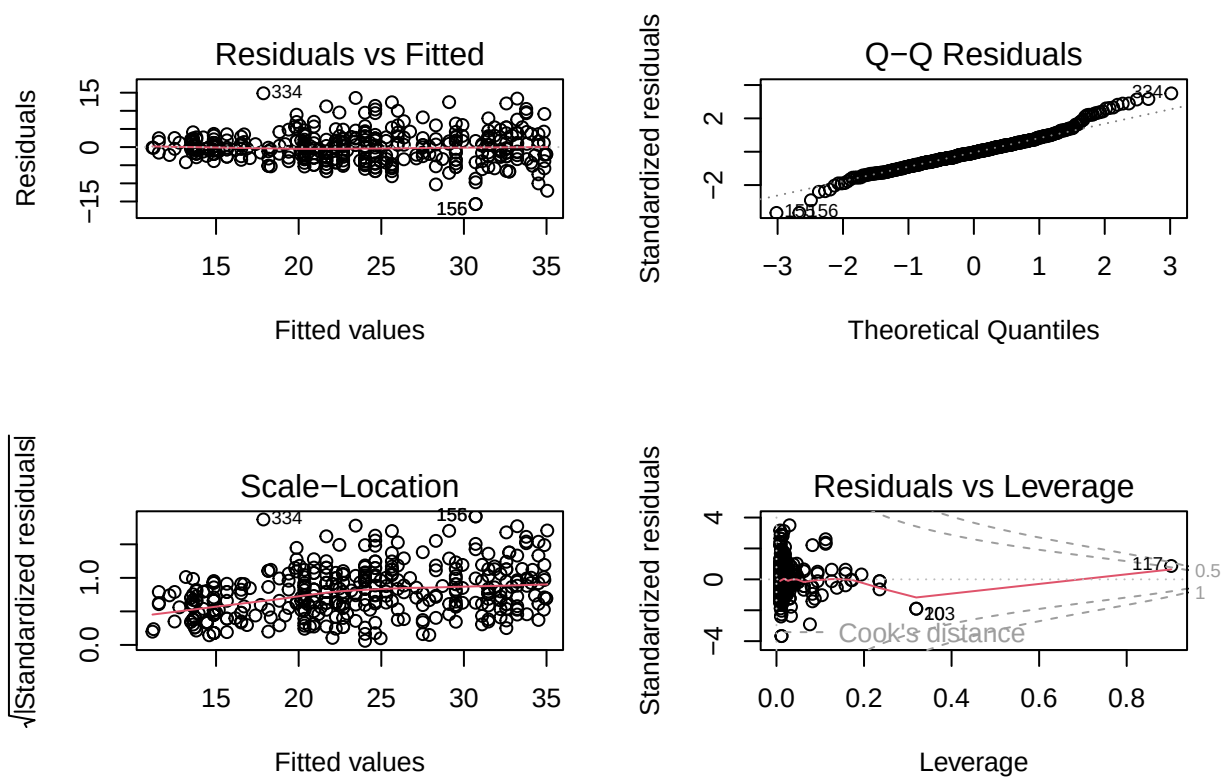
We'll discuss the remaining plots from plot(m1) and their utility in an upcoming lecture.

```
m2 = lm(mpg~poly(horsepower, degree = 10, raw = TRUE), data=Auto)
summary(m2)
```

```
##
## Call:
## lm(formula = mpg ~ poly(horsepower, degree = 10, raw = TRUE),
##     data = Auto)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15.7081  -2.5904  -0.1922   2.2859  14.8338
##
## Coefficients:
##                                          Estimate Std. Error t value
## (Intercept)                            -9.967e+02  3.324e+03  -0.300
## poly(horsepower, degree = 10, raw = TRUE)1   9.857e+01  3.156e+02   0.312
## poly(horsepower, degree = 10, raw = TRUE)2  -4.264e+00  1.306e+01  -0.327
## poly(horsepower, degree = 10, raw = TRUE)3   1.092e-01  3.101e-01   0.352
## poly(horsepower, degree = 10, raw = TRUE)4  -1.811e-03  4.689e-03  -0.386
```

```
## poly(horsepower, degree = 10, raw = TRUE)5   2.005e-05  4.719e-05   0.425
## poly(horsepower, degree = 10, raw = TRUE)6  -1.491e-07  3.206e-07  -0.465
## poly(horsepower, degree = 10, raw = TRUE)7   7.335e-10  1.453e-09   0.505
## poly(horsepower, degree = 10, raw = TRUE)8  -2.284e-12  4.212e-12  -0.542
## poly(horsepower, degree = 10, raw = TRUE)9   4.070e-15  7.058e-15   0.577
## poly(horsepower, degree = 10, raw = TRUE)10 -3.159e-18  5.201e-18  -0.607
##                                              Pr(>|t|)
## (Intercept)                                     0.764
## poly(horsepower, degree = 10, raw = TRUE)1      0.755
## poly(horsepower, degree = 10, raw = TRUE)2      0.744
## poly(horsepower, degree = 10, raw = TRUE)3      0.725
## poly(horsepower, degree = 10, raw = TRUE)4      0.700
## poly(horsepower, degree = 10, raw = TRUE)5      0.671
## poly(horsepower, degree = 10, raw = TRUE)6      0.642
## poly(horsepower, degree = 10, raw = TRUE)7      0.614
## poly(horsepower, degree = 10, raw = TRUE)8      0.588
## poly(horsepower, degree = 10, raw = TRUE)9      0.565
## poly(horsepower, degree = 10, raw = TRUE)10     0.544
##
## Residual standard error: 4.305 on 381 degrees of freedom
## Multiple R-squared:  0.7036, Adjusted R-squared:  0.6958
## F-statistic: 90.45 on 10 and 381 DF,  p-value: < 2.2e-16
```

```r
par(mfrow=c(2,2))
plot(m2)
```




Degree 10 shows an improvement over m1 because the plot shows a much straighter, flat red line. a degree of 2 might be a good choice due to the original parabolic shape, but the line remains curved with degree 2.